Syncner Troubleshooting & Maintenance Guide

📋 Table of Contents

1. Common Issues
2. Frontend Troubleshooting
3. Backend Troubleshooting
4. Database Issues
5. Performance Issues
6. Security Issues
7. Deployment Issues
8. Monitoring and Logging
9. Maintenance Procedures
10. Emergency Procedures

🚨 Common Issues

1. Authentication Issues

Problem: Users cannot log in

Symptoms:

- Login form shows "Invalid credentials" error
- Users are redirected to login page after successful login
- JWT token errors in browser console

Diagnosis:

```
# Check backend logs
curl -X POST https://your-backend-domain.onrender.com/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","password":"test123"}'

# Check JWT secret configuration
echo $JWT_SECRET
```

Solutions:

1. **Verify JWT Secret**:

   ```
   # Ensure JWT_SECRET is set in environment variables
   # Minimum 32 characters recommended
   JWT_SECRET=your-super-secure-jwt-secret-key-min-32-chars
   ```

2. **Check Password Hashing**:

```
// Verify password hashing in backend
const bcrypt = require('bcryptjs');
const isValid = await bcrypt.compare(password, hashedPassword);
```

3. **Clear Browser Storage**:

```
// Clear localStorage and sessionStorage
localStorage.clear();
sessionStorage.clear();
```

Problem: Session expires too quickly

Solutions:

1. **Adjust JWT Expiry**:

```
# Increase JWT_EXPIRE in environment variables
JWT_EXPIRE=7d  # 7 days instead of 30d
```

2. **Implement Token Refresh**:

```
// Add token refresh logic
const refreshToken = async () => {
  const response = await fetch('/api/auth/refresh', {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${token}` }
  });
  return response.json();
};
```

2. Database Connection Issues

Problem: Database connection fails

Symptoms:

- "Database connection failed" errors
- API endpoints return 500 errors
- Application crashes on startup

Diagnosis:

```
# Test MongoDB connection
mongosh "mongodb+srv://username:password@cluster.mongodb.net/syncner"

# Check connection string format
echo $MONGODB_URI
```

Solutions:

1. **Verify Connection String**:

```
# Ensure proper format
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/syncne
r?retryWrites=true&w=majority
```

2. **Check Network Access**:
   - Verify IP whitelist in MongoDB Atlas
   - Add `0.0.0.0/0` for Render deployment
   - Check firewall settings

3. **Test Connection**:

```
// Add connection test endpoint
app.get('/health/db', async (req, res) => {
  try {
    await mongoose.connection.db.admin().ping();
    res.json({ status: 'connected' });
  } catch (error) {
    res.status(503).json({ status: 'disconnected', error:
error.message });
  }
});
```

3. File Upload Issues

Problem: File uploads fail

Symptoms:

- "File upload failed" errors
- Files not appearing in uploads directory
- Large files rejected

Solutions:

1. **Check File Size Limits**:

```
// Increase file size limit
const multer = require('multer');
const upload = multer({
  limits: { fileSize: 50 * 1024 * 1024 }, // 50MB
  fileFilter: (req, file, cb) => {
    // Add file type validation
    const allowedTypes = ['image/jpeg', 'image/png',
'application/pdf', 'application/zip'];
    if (allowedTypes.includes(file.mimetype)) {
      cb(null, true);
    } else {
      cb(new Error('Invalid file type'), false);
    }
  }
});
```

2. **Check Storage Permissions**:

```
# Ensure uploads directory exists and is writable
mkdir -p uploads
chmod 755 uploads
```

3. **Add Upload Progress**:

```
// Add progress tracking
const uploadProgress = (req, res, next) => {
  let progress = 0;
  req.on('data', (chunk) => {
    progress += chunk.length;
    // Emit progress event
  });
  next();
};
```

🎨 Frontend Troubleshooting

1. Build Issues

Problem: Build fails with TypeScript errors

Solutions:

```
# Clear cache and reinstall
rm -rf node_modules package-lock.json
npm install
```

```
# Check TypeScript configuration
npx tsc --noEmit

# Fix type errors
npm run type-check
```

Problem: Build succeeds but app doesn't load

Solutions:

1. **Check Environment Variables**:

```
# Verify all required environment variables are set
echo $VITE_API_URL
echo $VITE_APP_NAME
```

2. **Check Browser Console**:

```
// Look for JavaScript errors
console.error('Check for runtime errors');
```

3. **Verify API Endpoints**:

```
// Test API connectivity
fetch(`${import.meta.env.VITE_API_URL}/health`)
  .then(response => response.json())
  .then(data => console.log(data));
```

2. State Management Issues

Problem: Redux state not updating

Solutions:

1. **Check Redux DevTools**:

```
// Install Redux DevTools extension
// Check if actions are being dispatched
```

2. **Verify Reducer Logic**:

```
// Ensure reducers are pure functions
const userReducer = (state = initialState, action) => {
  switch (action.type) {
    case 'SET_USER':
      return { ...state, user: action.payload }; // Don't mutate state
    default:
      return state;
  }
};
```

3. **Check Action Creators**:

```
// Ensure actions are properly created
const setUser = (user) => ({
  type: 'SET_USER',
  payload: user
});
```

3. Routing Issues

Problem: Routes not working correctly

Solutions:

1. **Check Route Configuration**:

```
// Verify route paths
<Route path="/student/dashboard" element={<StudentDashboard />} />
```

2. **Check Navigation**:

```
// Use proper navigation
import { useNavigate } from 'react-router-dom';
const navigate = useNavigate();
navigate('/student/dashboard');
```

3. **Check Protected Routes**:

```
// Verify authentication checks
const ProtectedRoute = ({ children }) => {
  const { isAuthenticated } = useAppSelector(state => state.auth);
  return isAuthenticated ? children : <Navigate to="/login" />;
};
```

⚙️ Backend Troubleshooting

1. API Endpoint Issues

Problem: API endpoints return 404

Solutions:

1. **Check Route Registration**:

```
// Ensure routes are properly registered
app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);
```

2. **Check Route Paths**:

```
// Verify route definitions
router.get('/users', getUsers);
router.post('/users', createUser);
```

3. **Check Middleware Order**:

```
// Ensure middleware is in correct order
app.use(cors());
app.use(express.json());
app.use('/api', routes);
```

Problem: API endpoints return 500 errors

Solutions:

1. **Check Error Handling**:

```
// Add proper error handling
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Something went wrong!' });
});
```

2. **Check Database Queries**:

```
// Add query validation
const getUsers = async (req, res) => {
```

```
    try {
      const users = await User.find(req.query);
      res.json(users);
    } catch (error) {
      console.error('Database error:', error);
      res.status(500).json({ error: 'Database error' });
    }
  };
```

2. Middleware Issues

Problem: CORS errors

Solutions:

```
// Configure CORS properly
const corsOptions = {
  origin: [
    'https://your-frontend-domain.vercel.app',
    'http://localhost:5173'
  ],
  credentials: true,
  optionsSuccessStatus: 200
};
app.use(cors(corsOptions));
```

Problem: Rate limiting too aggressive

Solutions:

```
// Adjust rate limiting
const rateLimit = require('express-rate-limit');
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 1000, // Increase limit
  message: 'Too many requests from this IP'
});
```

3. Authentication Middleware Issues

Problem: JWT verification fails

Solutions:

```
// Add better error handling
const protect = async (req, res, next) => {
```

```
  try {
    const token = req.header('Authorization')?.replace('Bearer ', '');

    if (!token) {
      return res.status(401).json({ error: 'No token provided' });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.id);

    if (!user) {
      return res.status(401).json({ error: 'User not found' });
    }

    req.user = user;
    next();
  } catch (error) {
    console.error('JWT verification error:', error);
    res.status(401).json({ error: 'Invalid token' });
  }
};
```

🗄️ Database Issues

1. Query Performance Issues

Problem: Slow database queries

Solutions:

1. **Add Indexes**:

```
// Create performance indexes
db.users.createIndex({ email: 1 });
db.userprogress.createIndex({ userId: 1, careerTrack: 1 });
```

2. **Optimize Queries**:

```
// Use projection to limit fields
const users = await User.find({ role: 'student' })
  .select('name email careerTrack')
  .lean(); // Returns plain objects
```

3. **Use Aggregation**:

```
// Use aggregation for complex queries
const stats = await UserProgress.aggregate([
```

```
    { $group: { _id: '$careerTrack', count: { $sum: 1 } } }
  ]);
```

2. Data Consistency Issues

Problem: Orphaned records

Solutions:

```
// Clean up orphaned records
const cleanupOrphanedRecords = async () => {
  // Find orphaned progress records
  const orphanedProgress = await UserProgress.find({
    userId: { $nin: await User.distinct('_id') }
  });

  if (orphanedProgress.length > 0) {
    await UserProgress.deleteMany({
      userId: { $nin: await User.distinct('_id') }
    });
    console.log(`Cleaned up ${orphanedProgress.length} orphaned records`);
  }
};
```

3. Connection Pool Issues

Problem: Too many database connections

Solutions:

```
// Configure connection pool
mongoose.connect(process.env.MONGODB_URI, {
  maxPoolSize: 10, // Maximum number of connections
  serverSelectionTimeoutMS: 5000, // Timeout for server selection
  socketTimeoutMS: 45000, // Socket timeout
  bufferMaxEntries: 0, // Disable mongoose buffering
  bufferCommands: false, // Disable mongoose buffering
});
```

⚡ Performance Issues

1. Slow Page Load Times

Problem: Frontend loads slowly

Solutions:

1. **Implement Code Splitting**:

```
// Lazy load components
const LazyComponent = React.lazy(() => import('./LazyComponent'));

// Use Suspense
<Suspense fallback={<Loading />}>
  <LazyComponent />
</Suspense>
```

2. **Optimize Bundle Size**:

```
// Analyze bundle
npm run build -- --analyze

// Remove unused dependencies
npm prune
```

3. **Implement Caching**:

```
// Add service worker for caching
if ('serviceWorker' in navigator) {
  navigator.serviceWorker.register('/sw.js');
}
```

2. API Response Times

Problem: API responses are slow

Solutions:

1. **Add Response Caching**:

```
// Cache API responses
const cache = require('memory-cache');

const getCachedData = async (key, fetchFunction) => {
  let data = cache.get(key);
  if (!data) {
    data = await fetchFunction();
    cache.set(key, data, 300000); // 5 minutes
  }
  return data;
};
```

2. **Optimize Database Queries**:

```
// Use efficient queries
const users = await User.find({ role: 'student' })
  .select('name email')
  .limit(50)
  .lean();
```

3. **Add Compression**:

```
// Enable gzip compression
const compression = require('compression');
app.use(compression());
```

3. Memory Usage Issues

Problem: High memory usage

Solutions:

1. **Monitor Memory Usage**:

```
// Add memory monitoring
setInterval(() => {
  const used = process.memoryUsage();
  console.log('Memory usage:', {
    rss: Math.round(used.rss / 1024 / 1024) + ' MB',
    heapTotal: Math.round(used.heapTotal / 1024 / 1024) + ' MB',
    heapUsed: Math.round(used.heapUsed / 1024 / 1024) + ' MB'
  });
}, 30000);
```

2. **Optimize Data Structures**:

```
// Use lean queries for read-only operations
const users = await User.find({}).lean();
```

3. **Implement Pagination**:

```
// Add pagination to large datasets
const getUsers = async (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 20;
  const skip = (page - 1) * limit;
```

```
    const users = await User.find({})
      .skip(skip)
      .limit(limit);

    res.json(users);
  };
```

🔒 Security Issues

1. Authentication Vulnerabilities

Problem: Weak password policies

Solutions:

```
// Implement strong password validation
const validatePassword = (password) => {
  const minLength = 8;
  const hasUpperCase = /[A-Z]/.test(password);
  const hasLowerCase = /[a-z]/.test(password);
  const hasNumbers = /\d/.test(password);
  const hasSpecialChar = /[!@#$%^&*(),.?":{}|<>]/.test(password);

  return password.length >= minLength &&
         hasUpperCase &&
         hasLowerCase &&
         hasNumbers &&
         hasSpecialChar;
};
```

Problem: JWT token exposure

Solutions:

```
// Secure token storage
const secureTokenStorage = {
  setToken: (token) => {
    // Store in httpOnly cookie instead of localStorage
    document.cookie = `token=${token}; httpOnly; secure; sameSite=strict`;
  },

  getToken: () => {
    // Retrieve from cookie
    return document.cookie
      .split('; ')
      .find(row => row.startsWith('token='))
      ?.split('=')[1];
```

```
  }
};
```

2. Input Validation Issues

Problem: SQL injection vulnerabilities

Solutions:

```
// Use parameterized queries
const getUserById = async (userId) => {
  // Good: Parameterized query
  return await User.findById(userId);

  // Bad: String concatenation
  // return await User.findOne({ _id: userId });
};
```

Problem: XSS vulnerabilities

Solutions:

```
// Sanitize user input
const xss = require('xss');

const sanitizeInput = (input) => {
  return xss(input, {
    whiteList: {
      p: [],
      br: [],
      strong: [],
      em: []
    }
  });
};
```

3. File Upload Security

Problem: Malicious file uploads

Solutions:

```
// Validate file types and content
const multer = require('multer');
const path = require('path');
```

```
const fileFilter = (req, file, cb) => {
  const allowedTypes = ['image/jpeg', 'image/png', 'application/pdf'];
  const allowedExtensions = ['.jpg', '.jpeg', '.png', '.pdf'];

  const fileExtension = path.extname(file.originalname).toLowerCase();

  if (allowedTypes.includes(file.mimetype) &&
      allowedExtensions.includes(fileExtension)) {
    cb(null, true);
  } else {
    cb(new Error('Invalid file type'), false);
  }
};
```

🚀 Deployment Issues

1. Build Failures

Problem: Frontend build fails on Vercel

Solutions:

1. **Check Build Logs**:

```
# Check Vercel build logs for errors
vercel logs --follow
```

2. **Verify Environment Variables**:

```
# Ensure all required environment variables are set
vercel env ls
```

3. **Check Dependencies**:

```
// Ensure all dependencies are in package.json
{
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  }
}
```

Problem: Backend deployment fails on Render

Solutions:

1. **Check Build Command**:

```
# Ensure build command is correct
npm install
```

2. **Check Start Command**:

```
# Ensure start command is correct
npm start
```

3. **Check Environment Variables**:

```
# Verify all environment variables are set in Render dashboard
```

2. Domain Configuration Issues

Problem: Custom domain not working

Solutions:

1. **Check DNS Configuration**:

```
# Verify DNS records
nslookup your-domain.com
```

2. **Check SSL Certificate**:

```
# Verify SSL certificate
openssl s_client -connect your-domain.com:443
```

3. **Check CORS Configuration**:

```
// Update CORS to include custom domain
const corsOptions = {
  origin: ['https://your-custom-domain.com'],
  credentials: true
};
```

📊 Monitoring and Logging

1. Application Monitoring

Setup Application Monitoring

```
// Add health check endpoints
app.get('/health', (req, res) => {
  res.json({
    status: 'healthy',
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    memory: process.memoryUsage(),
    version: process.env.npm_package_version
  });
});

app.get('/health/db', async (req, res) => {
  try {
    await mongoose.connection.db.admin().ping();
    res.json({ status: 'connected' });
  } catch (error) {
    res.status(503).json({ status: 'disconnected', error: error.message
});
  }
});
```

Setup Error Tracking

```
// Add error tracking
const winston = require('winston');

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.Console(),
    new winston.transports.File({ filename: 'error.log', level: 'error'
}),
    new winston.transports.File({ filename: 'combined.log' })
  ]
});

// Use logger in error handling
app.use((err, req, res, next) => {
  logger.error('Unhandled error:', err);
  res.status(500).json({ error: 'Internal server error' });
});
```

2. Performance Monitoring

Setup Performance Metrics

```
// Add performance monitoring
const performanceMiddleware = (req, res, next) => {
  const start = Date.now();

  res.on('finish', () => {
    const duration = Date.now() - start;
    logger.info('Request completed', {
      method: req.method,
      url: req.url,
      status: res.statusCode,
      duration: `${duration}ms`
    });
  });

  next();
};

app.use(performanceMiddleware);
```

🔧 Maintenance Procedures

1. Regular Maintenance Tasks

Daily Tasks

```
# Check application health
curl https://your-backend-domain.onrender.com/health

# Check database connectivity
curl https://your-backend-domain.onrender.com/health/db

# Monitor error logs
tail -f logs/error.log
```

Weekly Tasks

```
# Update dependencies
npm update

# Run security audit
npm audit
```

```
# Clean up old logs
find logs/ -name "*.log" -mtime +7 -delete
```

Monthly Tasks

```
# Database maintenance
node scripts/cleanup-orphaned-records.js

# Performance analysis
node scripts/analyze-performance.js

# Security review
npm audit fix
```

2. Database Maintenance

Regular Cleanup

```
// Cleanup script
const cleanupDatabase = async () => {
  try {
    // Remove old attendance records (older than 1 year)
    const oneYearAgo = new Date();
    oneYearAgo.setFullYear(oneYearAgo.getFullYear() - 1);

    const result = await Attendance.deleteMany({
      date: { $lt: oneYearAgo }
    });

    console.log(`Cleaned up ${result.deletedCount} old attendance
records`);

    // Remove orphaned progress records
    const orphanedProgress = await UserProgress.find({
      userId: { $nin: await User.distinct('_id') }
    });

    if (orphanedProgress.length > 0) {
      await UserProgress.deleteMany({
        userId: { $nin: await User.distinct('_id') }
      });
      console.log(`Cleaned up ${orphanedProgress.length} orphaned progress
records`);
    }

  } catch (error) {
    console.error('Database cleanup failed:', error);
```

```
  }
};
```

Index Maintenance

```
// Rebuild indexes
const rebuildIndexes = async () => {
  try {
    const collections = ['users', 'userprogress', 'curriculum',
'attendance'];

    for (const collection of collections) {
      await mongoose.connection.db.collection(collection).reIndex();
      console.log(`Rebuilt indexes for ${collection}`);
    }

  } catch (error) {
    console.error('Index rebuild failed:', error);
  }
};
```

3. Backup Procedures

Automated Backups

```
// Backup script
const backupDatabase = async () => {
  const { exec } = require('child_process');
  const date = new Date().toISOString().split('T')[0];

  const command = `mongodump --uri="${process.env.MONGODB_URI}" --
out="./backups/backup_${date}"`;

  return new Promise((resolve, reject) => {
    exec(command, (error, stdout, stderr) => {
      if (error) {
        console.error('Backup failed:', error);
        reject(error);
      } else {
        console.log('Backup completed successfully');
        resolve(stdout);
      }
    });
  });
};

// Schedule daily backups
const cron = require('node-cron');
```

```
cron.schedule('0 2 * * *', () => {
  backupDatabase();
});
```

Backup Verification

```
// Verify backup integrity
const verifyBackup = async (backupDate) => {
  const { exec } = require('child_process');

  const command = `mongorestore --dryRun --
uri="${process.env.MONGODB_URI}" ./backups/backup_${backupDate}`;

  return new Promise((resolve, reject) => {
    exec(command, (error, stdout, stderr) => {
      if (error) {
        console.error('Backup verification failed:', error);
        reject(error);
      } else {
        console.log('Backup verification successful');
        resolve(stdout);
      }
    });
  });
};
```

🚨 Emergency Procedures

1. System Outage Response

Immediate Response

1. **Check System Status**:

```
# Check frontend
curl https://your-frontend-domain.vercel.app

# Check backend
curl https://your-backend-domain.onrender.com/health

# Check database
curl https://your-backend-domain.onrender.com/health/db
```

2. **Check Logs**:

```

```

```
# Check application logs
tail -f logs/error.log

# Check system logs
journalctl -u your-service -f
```

3. **Restart Services**:

```
# Restart backend service
# (This depends on your hosting platform)
```

Escalation Procedures

1. **Level 1**: Check logs and restart services
2. **Level 2**: Check database connectivity and performance
3. **Level 3**: Check external service dependencies
4. **Level 4**: Contact hosting provider support

2. Data Recovery Procedures

Database Recovery

```
# Restore from backup
mongorestore --
uri="mongodb+srv://username:password@cluster.mongodb.net/syncner"
./backups/backup_2024-01-15

# Verify data integrity
node scripts/verify-data-integrity.js
```

File Recovery

```
# Restore uploaded files
tar -xzf uploads_backup_2024-01-15.tar.gz

# Verify file integrity
find uploads/ -type f -exec md5sum {} \;
```

3. Security Incident Response

Immediate Actions

1. **Isolate Affected Systems**:

```
# Block suspicious IPs
iptables -A INPUT -s suspicious-ip -j DROP
```

2. **Change Credentials**:

```
# Change database passwords
# Change API keys
# Invalidate JWT tokens
```

3. **Preserve Evidence**:

```
# Save logs
cp /var/log/application.log ./incident-logs/

# Save system state
ps aux > ./incident-logs/processes.txt
netstat -tulpn > ./incident-logs/network.txt
```

Investigation Steps

1. **Analyze Logs**:

```
# Search for suspicious activity
grep "suspicious-pattern" logs/access.log
```

2. **Check System Integrity**:

```
# Verify file integrity
find /app -type f -exec md5sum {} \;
```

3. **Document Findings**:
    - Create incident report
    - Document timeline
    - Identify root cause
    - Plan remediation

4. Performance Emergency Response

Immediate Actions

1. **Scale Resources**:

```
# Increase server resources if possible
# Enable CDN caching
# Implement rate limiting
```

2. **Optimize Queries**:

```
// Add query optimization
const optimizedQuery = async () => {
  return await User.find({})
    .select('name email')
    .limit(100)
    .lean();
};
```

3. **Enable Maintenance Mode**:

```
// Add maintenance mode
const maintenanceMode = (req, res, next) => {
  if (process.env.MAINTENANCE_MODE === 'true') {
    return res.status(503).json({
      message: 'System under maintenance',
      estimatedTime: '30 minutes'
    });
  }
  next();
};
```

📞 Support Contacts

Internal Support

- **Technical Lead**: tech-lead@syncner.com
- **Database Admin**: db-admin@syncner.com
- **Security Team**: security@syncner.com

External Support

- **Vercel Support**: https://vercel.com/support
- **Render Support**: https://render.com/support
- **MongoDB Atlas Support**: https://support.mongodb.com

Emergency Contacts

- **24/7 Support**: +1-555-EMERGENCY
- **Critical Issues**: critical@syncner.com
- **Security Incidents**: security-incident@syncner.com

Troubleshooting Guide Version: v1.0
Submitted On: September 2025